API Error Handling: 4xx vs 5xx & Problem Details

errors clients can act on, and which ones to retry

Watch the 4-min video

> **In one line:** pick the right status class (4xx = client's fault, 5xx = server's fault), return one consistent Problem Details body with a stable machine-readable code and a trace ID, retry only transient errors, and never leak internals.

1 Why error design matters

Imagine a signup API that fails in three different ways. One endpoint returns `200` with `"error": true` hidden in the body. Another returns `500` with a stack trace that reveals the database host. A third returns plain text: “Something went wrong”.

No client can handle these reliably. Errors are part of your API contract, just like successful responses. Good error design gives every client the same three answers: **what kind of failure** (the status code), **exactly which error** (a stable code) and **what to do next** (fix, retry or contact support).

200 OK (hiding an error)

```
{
  "error": true,
  "msg": "failed"
}
```

monitoring sees success

500 (leaking internals)

```
NullPointerException
  at UserRepo.save
  host db-prod-3:5432
  hibernate 5.4.2
```

a gift to attackers

500 text/plain

```
Something went wrong
```

nothing to switch on

three endpoints, three error styles: no client can handle these reliably

Figure 1. Three common anti-patterns. Each one breaks something different: monitoring, security, or client logic.

2 Start with the right status class

The first digit tells the client whose fault it is. A **4xx** means the client sent something wrong, so resending the same request won't help. A **5xx** means the server failed, and the same request might work later.

Code	Meaning	Typical cause
400 Bad Request	Malformed request	Invalid JSON, missing required field
401 Unauthorized	Not authenticated	Missing or expired token
403 Forbidden	Authenticated, but not allowed	User lacks the role
404 Not Found	No such resource	Wrong ID, or hidden from this caller
409 Conflict	Clashes with current state	Duplicate email, version mismatch
422 Unprocessable Content	Well-formed but invalid content	End date before start date
429 Too Many Requests	Rate limited	Over the limit; see `Retry-After`
500 Internal Server Error	Unexpected server failure	Bug, unhandled exception
502 Bad Gateway	Proxy got a bad upstream answer	Backend crashed or hung up

503 Service Unavailable	Temporarily can't serve	Overloaded, no healthy backends
504 Gateway Timeout	Proxy gave up waiting on upstream	Slow backend or database

Never return 200 with an error inside. Monitoring counts error rates by status code, caches and CDNs may store a 200, and retry logic in clients and proxies looks only at the code. A 200 error is invisible to all of them.

3 One error envelope: Problem Details (RFC 9457)

Use the same error body everywhere. The standard is **Problem Details for HTTP APIs**, **RFC 9457**, which replaced the older RFC 7807. Its content type is `application/problem+json`. It defines five fields, and you may add your own extension fields, such as a machine-readable `code` and a list of field errors.

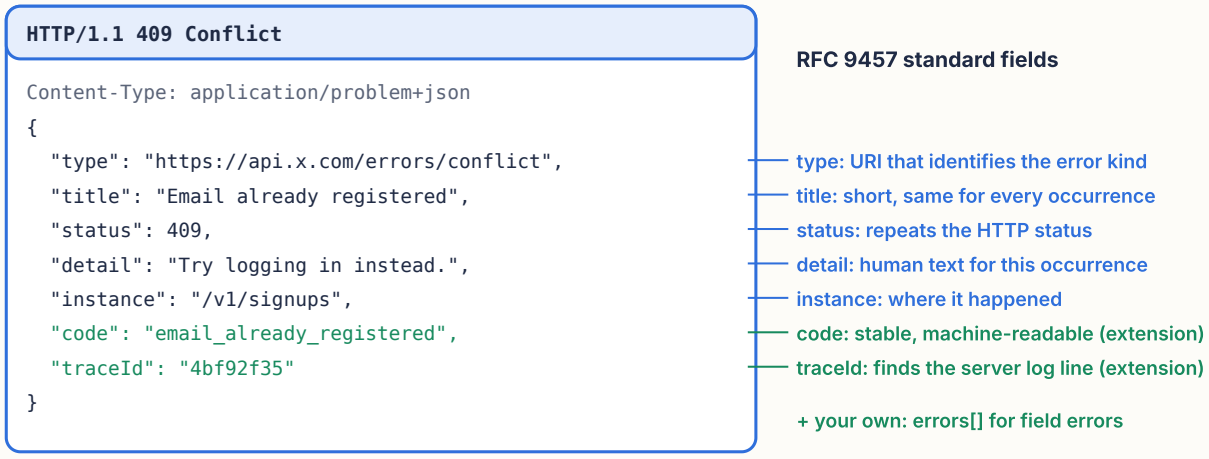


Figure 2. A Problem Details response for a duplicate signup. Blue: standard fields. Green: extension fields you add.

WORKED EXAMPLE: DUPLICATE SIGNUP

A user signs up with an email that already exists. Return **409 Conflict** with `"code": "email_already_registered"` and a friendly `detail` like “This email is already registered. Try logging in instead.”

The mobile app **switches on the code**, never on the message text: `if code == "email_already_registered"` → show the login screen. So you can reword or translate the message into Hindi or German tomorrow without breaking any client.

For a form with several bad fields, return them all in one response (status 422), so the user fixes everything in one round trip:

```
"errors": [
  { "field": "email",    "code": "invalid_format" },
  { "field": "password", "code": "too_short", "min": 12 }
]
```

4 Which errors should a client retry?

Retry only **transient** failures: problems that may disappear on their own. Always retry with exponential backoff and jitter, and with a cap on attempts, so retries don't pile onto a struggling server.

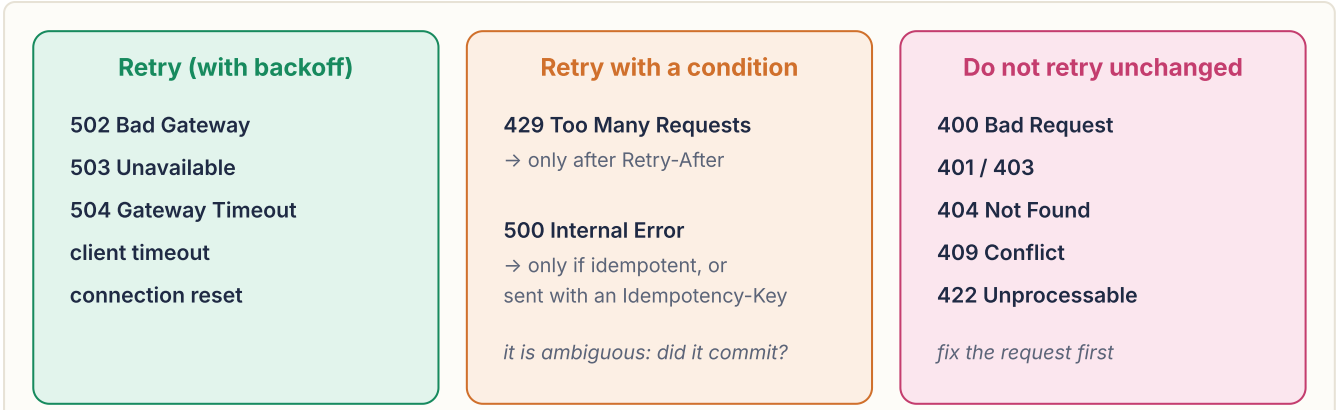


Figure 3. Retry rules. A 500 is ambiguous: the server may have done the work before failing, so a blind retry of a payment could charge twice.

- **Retry:** 502, 503, 504, timeouts and connection resets. These are usually a brief overload, deploy or network blip.
- **Retry 429 only after waiting** for the `Retry-After` delay. Retrying immediately makes the overload worse.
- **500 is ambiguous.** Retry it only if the operation is idempotent (GET, PUT, DELETE) or protected by an idempotency key.
- **Never blindly retry other 4xx** errors such as 400, 403 or 422. The same request will fail the same way. (401 is the one case where the client may refresh its token once and try again.)

5 Don't leak internals, and keep it consistent

Stack traces, SQL errors, hostnames and library versions in responses are a gift to attackers: they reveal your stack and which known vulnerabilities to try. Instead, **log the full details on the server with a trace ID**, and return only that ID to the client. When a user reports "trace 4bf92f35", support finds the exact log line in seconds.

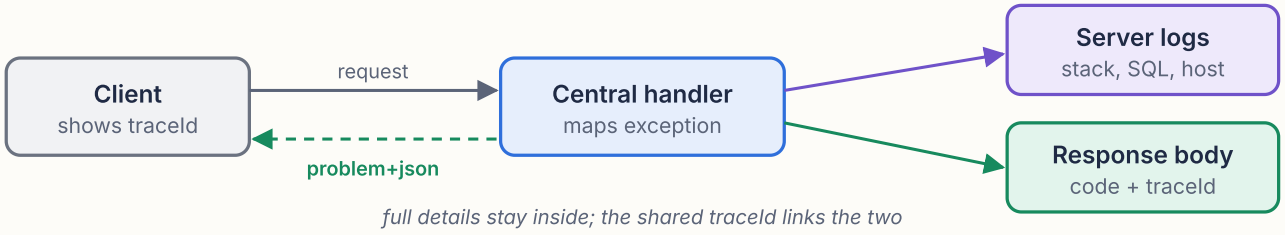


Figure 4. One central handler catches every exception, logs the full details, and returns a clean Problem Details body carrying the same trace ID.

Watch for existence leaks. If a caller asks for `/invoices/981` that belongs to another customer, a 403 confirms the invoice exists. Returning **404** hides that it exists at all. The same applies to login: "wrong email or password" instead of "no such email".

A few more habits:

- **Return all field errors at once**, not one per round trip.
- **Document every error code** in your API reference and keep codes stable. Renaming a code is a breaking change, like renaming a field.
- **Map exceptions to responses in one central handler** (middleware or an exception filter), so every endpoint speaks the same error language.

COMMON MISTAKES

- Returning 200 with `"success": false`: dashboards show 0% errors during an outage.
- Clients parsing the `detail` or `message` text instead of a stable code.
- Returning 500 for bad input. Validation failures are 400 or 422, and a flood of 500s pages the on-call engineer for a client bug.
- Retrying every error, including 400 and 403, or retrying a non-idempotent POST after a 500.

6 Key terms

4xx: client error: fix the request

5xx: server error: may work later

Problem Details: RFC 9457 error format

application/problem+json: its content type

Error code: stable machine-readable string

Trace ID: links a response to its log line

Transient error: may succeed on retry (502/503/504)

Retry-After: how long to wait before retrying

Existence leak: 403 reveals a hidden resource

7 Interview questions

Q1. What's the difference between a 4xx and a 5xx? BASIC

4xx: the client sent something wrong (400, 403, 404, 409, 422, 429); resending it unchanged won't help. 5xx: the server failed (500, 502, 503, 504); it may work later.

Q2. What should an error response contain? BASIC

The right status plus a Problem Details body: type, title, status, detail, instance, and extensions like a stable `code`, field errors and a trace ID. No stack traces or SQL.

Q3. A client calls your payment API and gets a 500. Should it retry? SCENARIO

Only if idempotent. The POST may have charged the card before failing, so a blind retry risks a double charge. With an `Idempotency-Key`, the retry returns the first attempt's stored result.

Q4. Your SDK should retry automatically. Which errors, and how? SCENARIO

502, 503, 504, timeouts and connection resets, with exponential backoff and jitter (say 200 ms, 400 ms, 800 ms, max 3 tries). 429 only after `Retry-After`. 500 only for idempotent calls. Never 400, 403, 404 or 422.

Q5. Is rewording an error message a breaking change? SCENARIO

Not if clients switch on the stable `code` field. The `detail` text is for humans and can change or be translated freely. Renaming the code, though, is breaking and needs a new version or deprecation.

Q6. A user requests another tenant's invoice. 403 or 404? TRICKY

Usually 404. A 403 confirms the invoice exists, which leaks information (an attacker can enumerate IDs). Return 404 for resources the caller isn't allowed to know about, and 403 when they can see it exists but can't perform this action.

Q7. 400 or 422 for a validation failure? TRICKY

400 for a request the server can't parse (broken JSON, wrong types). 422 for well-formed content that fails business rules (end date before start date). Many APIs use 400 for both; what matters is being consistent and returning field-level codes.

Next: SD-C-012 · Rate Limiting Algorithms backendroadmap.com